Solution: Next Greater Element

Problem Statement

Given an array, print the Next Greater Element (NGE) for every element.

The Next Greater Element for an element `x` is the first greater element on the right side of `x` in the array.

Elements for which no greater element exist, consider the next greater element as -1.

Examples

Example 1:

```
 Input: [4, 5, 2, 25]
 Output: [5, 25, 25, -1]
 Explanation: The NGE for 4 is 5, 5 is 25, 2 is 25, and there is no NGE
for 25.
```

Example 1:

```
 Input: [13, 7, 6, 12]
 Output: [-1, 12, 12, -1]
```

Example 1:

```
 Input: [1, 2, 3, 4, 5]
 Output: [2, 3, 4, 5, -1]
```

Solution:

A simple algorithm is to run two loops: the outer loop picks all elements one by one, and the inner loop looks for the first greater element for the element picked by the outer loop. However, this algorithm has a time complexity of $O(n^2)$.

We can use a more optimized approach using Stack data structure. The algorithm will leverage the nature of the stack data structure, where the most recently added (pushed) elements are the first ones to be removed (popped). Starting from the end of the array, the algorithm always maintains elements in the stack that are larger than the current element. This way, it ensures that it has a candidate for the "next larger element". If there is no larger element, it assigns -1 to that position. It handles each element of the array only once, making it an efficient solution.

Detailed Step-by-Step Walkthrough

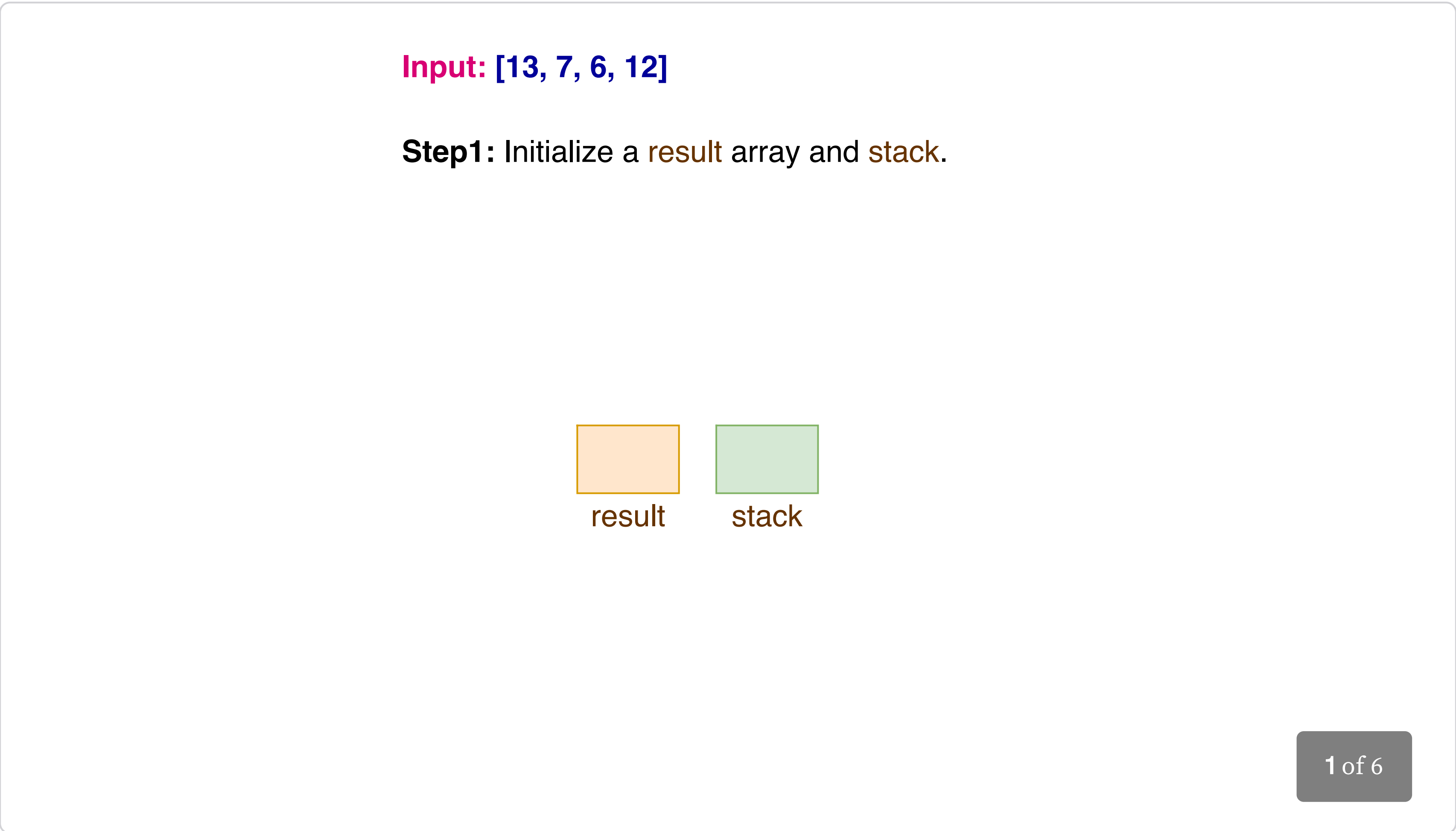
1. The function receives an array `arr`.
2. Initialize an empty stack `s` and an output array `res` of size equal to the input array, with all elements initialized to -1. `res` will store the result, i.e., the next larger element for each position in the array.
3. Start a loop that goes from the last index of the array to the first (0 index).
4. In each iteration, while there are elements in the stack and the top element of the stack is less than or equal to the current element in the array, remove elements from the stack. This step ensures that we retain only the elements in the stack that are larger than the current element.
5. After the popping process, if there is still an element left in the stack, it is the next larger element for the current array element. So, assign the top element of the stack to the corresponding position in the `res` array.
6. Now, push the current array element into the stack. This action considers the current element as a possible "next larger element" for the upcoming elements in the remaining iterations.
7. Repeat steps 4-6 for all the elements of the array.
8. At the end of the loop, `res` will contain the next larger element for each position in the array. Return this array `res`.

Algorithm Walkthrough

Let's consider the input and observe how above algorithm works.

1. **Initialize Data Structures**:
    - **Input Array**: `[13, 7, 6, 12]`
    - **Result Array**: `[0, 0, 0, 0]` (Initially set to zeros)
    - **Stack**: Empty (Will store elements during iteration)
2. **Processing Each Element** (**Reverse Order**):
    - The algorithm processes the array from right to left.
3. **Last Element** (**Value 12**):
    - Stack is empty, indicating no greater element for 12.
    - **Result Array**: `[0, 0, 0, -1]` (Updates the last position to -1)
    - Push element 12 onto the stack.
4. **Third Element** (**Value 6**):
    - Stack's top element is 12, which is greater than 6.
    - **Result Array**: `[0, 0, 12, -1]` (Updates the value at the third position to 12)
    - Push element 6 onto the stack.
5. **Second Element** (**Value 7**):
    - Stack's top element is 6, which is less than 7, so it's popped.
    - Next, the stack's top element is 12, which is greater than 7.
    - **Result Array**: `[0, 12, 12, -1]` (Updates the value at the second position to 12)
    - Push element 7 onto the stack.
6. **First Element** (**Value 13**):
    - Stack's top element is 7, which is less than 13, so it's popped.
    - Next, stack's top element is 12, which is also less than 13, so it's popped.
    - Stack is now empty, indicating no greater element for 13.
    - **Result Array**: `[-1, 12, 12, -1]` (Updates the first position to -1)
    - Push element 13 onto the stack.

Here is the visual representation of the algorithm:

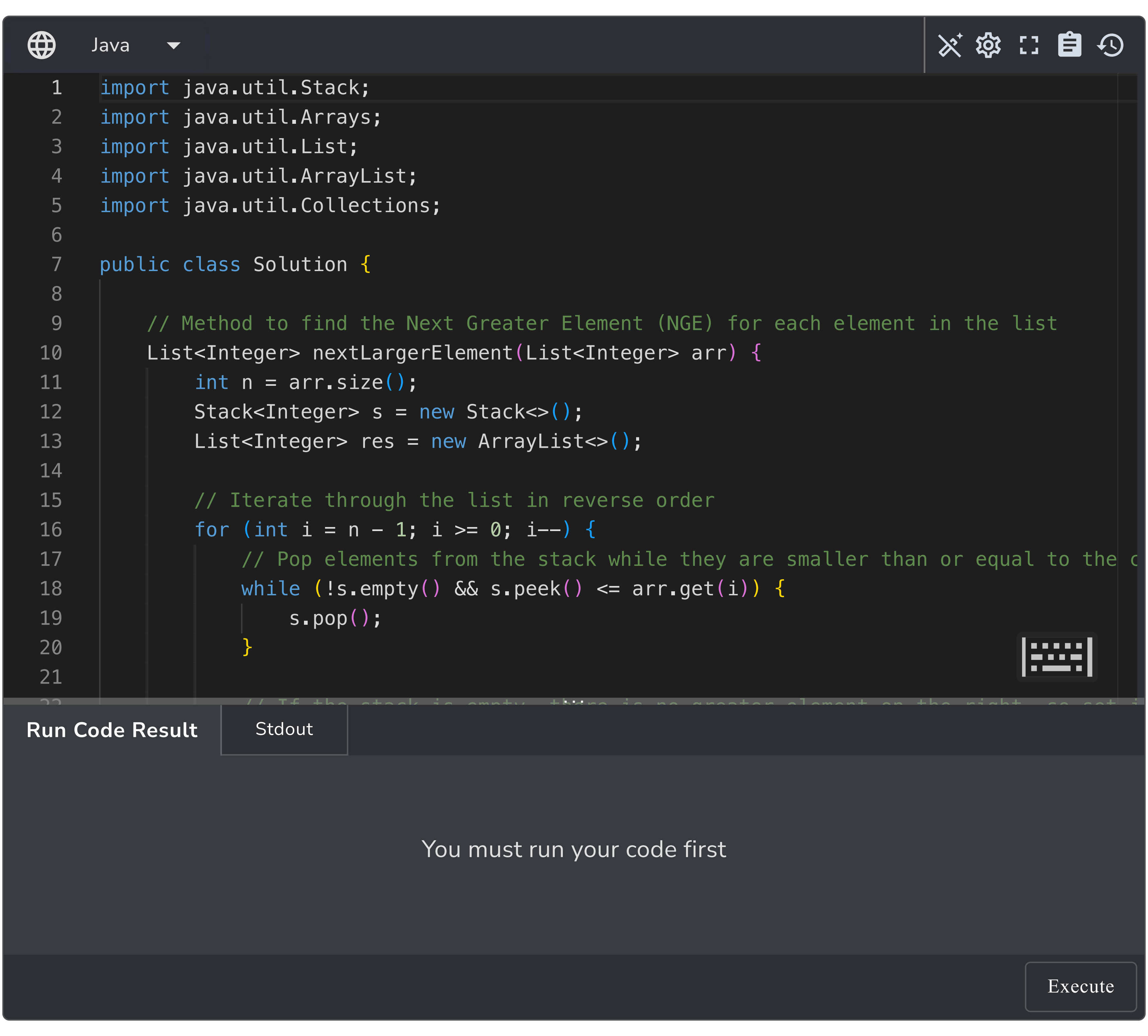


Input: [13, 7, 6, 12]

Step1: Initialize a result array and stack.

Code

Here is the code for this algorithm:



```java
import java.util.Stack;
import java.util.Arrays;
import java.util.List;
import java.util.ArrayList;
import java.util.Collections;

public class Solution {

    // Method to find the Next Greater Element (NGE) for each element in the list
    List<Integer> nextLargerElement(List<Integer> arr) {
        int n = arr.size();
        Stack<Integer> s = new Stack<>();
        List<Integer> res = new ArrayList<>();

        // Iterate through the list in reverse order
        for (int i = n - 1; i >= 0; i--) {
            // Pop elements from the stack while they are smaller than or equal to the c
            while (!s.empty() && s.peek() <= arr.get(i)) {
                s.pop();
            }

```

You must run your code first

Time and Space Complexity

Time Complexity: The worst-case time complexity of this algorithm is $O(n)$ as every element is pushed and popped from the stack exactly once.

Space Complexity: The space complexity of this algorithm is $O(n)$ as we are using a stack and an array to store the next greater elements.

← Previous
Problem 4: Next Greater Element

Next →
Problem 5: Sorting A Stack

☐ Mark as Completed